

ר

JavaScript

```
setCount(prevCount => prevCount + 1);
```

이 줄이 왜 이렇게 써야 하는지, 그리고 왜 아래처럼 쓰면 안 되는지까지 완전 자세히 설명해드릴게요.

전체 코드에서 일어나는 일 간단 정리

이 컴포넌트는 단순한 카운터예요:

- 현재 count 값 보여주기
- • 버튼 누르면 1 증가 (원래는 2번 증가시키려는 의도)
- • 버튼 누르면 1 감소

근데 여기서 + 버튼에 문제가 있어요!

문제의 코드 (+ 버튼)

```
JavaScript

<Button
  title="+"
  onPress={() => {
    setCount(prevCount => prevCount + 1); // 올바른 방법
    setCount(prevCount => prevCount + 1); // 올바른 방법
  }}
/>
```

틀린 방법 (많은 초보자들이 이렇게 씀)

```
JavaScript

onPress={() => {
  setCount(count + 1); // X 위험함!
  setCount(count + 1); // X 또 위험함!
}}
```

왜 `count + 1`은 위험한가? → 상태가 "배치 업데이트"되기 때문

리액트는 성능 최적화를 위해 한 번의 클릭(이벤트) 안에서 일어나는 여러 개의 `setState` (또는 `setCount`)를 모아서 한 번에 처리해요. 이걸 배치 업데이트(batch update)라고 해요.

그래서 위의 잘못된 코드를 실행하면:

1. 버튼 클릭 → `onPress` 실행
2. 첫 번째 `setCount(count + 1)` → "나중에 count를 현재 count + 1로 바꿔줘" 예약
3. 두 번째 `setCount(count + 1)` → 또 "나중에 count를 현재 count + 1로 바꿔줘" 예약
4. 리액트가 두 개를 모아서 한 번에 처리 → 둘 다 같은 `count` 값을 보고 있음 (클릭 당시의 값)
5. 결과: count가 1만 증가함 (+2가 안 됨!)

예를 들어 count가 5였다면:

- 잘못된 방식 → 5 → 6 (한 번만 증가)
- 올바른 방식 → 5 → 7 (두 번 증가)

그래서 등장하는 게 함수형 업데이트!

JavaScript

```
setCount(prevCount => prevCount + 1);
```

이렇게 쓰면 리액트가 이전 상태 값을 정확히 보장해줘요.

동작 원리

리액트가 내부적으로 이렇게 처리해요:

1. 첫 번째 setCount → 이전 상태(5)를 받아서 $5 + 1 = 6$ 반환
2. 두 번째 setCount → 첫 번째 업데이트가 적용된 상태(6)를 받아서 $6 + 1 = 7$ 반환
3. 최종적으로 count = 7로 업데이트!

즉, 업데이트가 순서대로 체이닝되면서 정확히 두 번 증가하는 거예요.

요약: 언제 이렇게 써야 할까?

상황	추천 방식	이유
현재 상태 값에 의존해서 업데이트	<code>setCount(prev => prev + 1)</code>	정확한 최신 상태 보장
상태를 여러 번 연속으로 변경	함수형 업데이트 필수	배치 업데이트 때문에
단순히 고정값으로 변경	<code>setCount(100)</code>	상관 없음

보너스: 더 깔끔하게 +2 해주는 방법

JavaScript

```
onPress={() => {  
  setCount(prev => prev + 2); // 한 번에 +2  
}}
```

또는 정확히 두 번 증가 보장하면서 깔끔하게:

JavaScript

```
onPress={() => {  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
}}
```

결론

JavaScript

```
setCount(prevCount => prevCount + 1);
```

이 코드는 리액트에서 상태를 안전하고 정확하게 업데이트하는 가장 중요한 패턴 중 하나예요.

특히 카운터, 장바구니 수량, 투두 완료 개수 등 연속으로 상태를 변경할 때는 반드시 이렇게 쓰는 습관을 들이는 게 좋아요!

이해 되셨나요? 더 궁금한 부분 있으면 언제든지 물어보세요!

prevCount는 어디서 왔지?

정말 좋은 질문이에요!!

`prevCount` 는 내가 지은 이름이 아니라, **리액트가 알아서 넣어주는 매직 변수**예요.

prevCount는 누가, 언제, 어떻게 주는 걸까?

`setCount` 를 일반 값 대신에 **함수**로 호출하면,
리액트가 그 함수를 실행할 때 **자동으로** 최신 상태 값을 첫 번째 인자로 넘겨줍니다.

JavaScript

```
setCount( prevCount => { ... } )  
//           ↑↑↑↑↑↑↑↑  
//           리액트가 이 자리에 값을 넣어줌!
```

즉, 실제로 리액트 내부에서는 이렇게 실행되는 거예요:

JavaScript

```
// 1번째 호출될 때  
prevCount => prevCount + 1    // ← 리액트가 prevCount에 5를 넣어줌 → 6 반환  
  
// 2번째 호출될 때 (첫 번째 업데이트가 반영된 후!)  
prevCount => prevCount + 1    // ← 리액트가 prevCount에 6을 넣어줌 → 7 반환
```

이름은 내가 맘대로 지을 수 있어요!

`prevCount` 라는 이름은 그냥 관례이고, 사실 이렇게 다 써도 돼요:

JavaScript

```
setCount(current => current + 1);  
setCount(c => c + 1);  
setCount(prev => prev + 1);  
setCount(count => count + 1);  
setCount(x => x + 1); // 심지어 x도 됨!
```

모두 동일하게 동작해요!

(보통은 `prev`, `current`, `c` 중 하나를 많이 써요)

다시 정리하면

질문	답변
<code>prevCount</code> 는 어디서 왔나요?	리액트가 <code>setCount(함수)</code> 를 호출할 때 자동으로 넣어주는 최신 상태 값
누가 넣어주나요?	리액트 자체가 넣어줌 (우리가 <code>import</code> 하거나 선언한 게 아님)
이름은 정해져 있나요?	아니요! 첫 번째 파라미터 이름은 내가 자유롭게 지을 수 있음
언제 이 값을 받을 수 있나요?	<code>setCount(값)</code> → 안 됨 <code>setCount(함수)</code> → 됨!

한 줄 요약

JavaScript

```
setCount(prev => prev + 1)
```

→ 이 화살표 함수의 `prev` 라는 파라미터는

리액트가 "지금 이 순간의 진짜 최신 `count` 값"을 넣어주는 선물 박스예요

이제 완전히 이해 되셨죠?

이 개념만 확실히 알면 리액트 상태 업데이트 버그 90%는 사라집니다!

UseState에 함수 넣으면 최신값이 자동으로 들어가?

네, 정확히 맞아요!

결론부터 말하면:

useState의 setter에 함수를 넣으면 → 리액트가 자동으로 “가장 최신 상태값”을 그 함수의 첫 번째 인자로 넣어줍니다.

공식 문서에 나온 정확한 규칙

tsx

```
const [state, setState] = useState(initialValue);
```

// 1. 일반 방식 (고정값이나 현재 변수 사용)

```
setState(100); // 무조건 100으로 바꿈
```

```
setState(count + 1); // 현재 렌더링 시점의 count 기준 (배치 업데이트 시 위험)
```

// 2. 함수형 업데이트 (이게 진짜 최신값 보장!)

```
setState(prevState => {
```

```
// prevState ← 여기 들어오는 게 "정말로 최신" 상태값!
return prevState + 1;
});
```

언제 최신값이 필요한가? (실제 예시 4가지)

상황	잘못된 코드 (버그 발생)	올바른 코드 (항상 정확)
한 번 클릭에 2번 이상 증가	<pre>setCount(count + 1); setCount(count + 1); → +1만 됨</pre>	<pre>setCount(c => c + 1); setCount(c => c + 1); → +2 정확</pre>
비동기 API 받은 후 상태 업데이트	버튼 누르고 3초 뒤에 <pre>setCount(count + 1)</pre> → 오래된 count 사용 가능	<pre>setCount(c => c + 1)</pre> → 3초 뒤에도 최신값 보장
setTimeout / setInterval 안에서	<pre>setInterval(() => setCount(count + 1), 1000)</pre> → 첫 번째 count만 계속 씬	<pre>setInterval(() => setCount(c => c + 1), 1000)</pre> → 진짜 1초마다 증가
동시에 여러 상태 업데이트	여러 버튼이 동시에 count 바꿀 때	함수형 업데이트면 순서 보장

리액트가 내부적으로 하는 일 (간단 그림)

JavaScript

```
// 우리가 쓴 코드
setCount(prev => prev + 1);
setCount(prev => prev + 1);

// 리액트가 실제로 실행하는 순서
prev => prev + 1    // prev = 5 → 반환 6
prev => prev + 1    // prev = 6 → 반환 7    최신값이 체인처럼 이어짐!
```

한 줄 요약 (이거만 기억하세요)

setState(값) → 내가 준 값으로 바꿔

setState(함수) → 리액트가 최신값을 함수 안에 넣어주고, 내가 반환한 값으로 바꿔줌

그래서 실무에서는 거의 90% 이상 이렇게 씁니다:

JavaScript

```
// 실무에서 가장 많이 보는 패턴들
setCount(c => c + 1);
setCount(c => c - 1);
setCount(c => c + amount);
```

```
setUser(prev => ({ ...prev, name: "새이름" }));  
setList(prev => [...prev, newItem]);
```

이제 완벽하게 이해하셨죠?

이게 리액트 상태 관리의 핵심 중 핵심이에요!

앞으로 카운터 만들 때 절대 `count + 1` 쓰지 마세요 → `c => c + 1` 만 쓰세요!

함수형 업데이트 실습 예제 보여줘

물론이죠!

지금 바로 5가지 실습 예제를 직접 실행해볼 수 있게 만들어줄게요.

코드만 복사해서 App.js에 붙여넣으면 바로 동작합니다!

실습 1: +1 버튼이 진짜 2번 증가하는지 확인 (핵심 실습!)

```
jsx  
  
import React, { useState } from 'react';  
import { View, Text, Button } from 'react-native';  
  
export default function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>  
      <Text style={{ fontSize: 50, marginBottom: 50 }}>{count}</Text>  
  
      {/* 잘못된 방법: 한 번만 증가함 */}  
      <Button  
        title="잘못된 방법 (+2 하려다 +1만 됨)"  
        onPress={() => {  
          setCount(count + 1);  
          setCount(count + 1);  
          // 리액트가 배치 처리해서 둘 다 같은 count 값을 봄 → +1만 됨  
        }}  
      />  
  
      <View style={{ height: 20 }} />  
  
      {/* 올바른 방법: 진짜 +2 됨! */}  
      <Button  
        title="올바른 방법 (함수형 업데이트 → 진짜 +2)"  
        color="red"  
        onPress={() => {  
          setCount(prev => prev + 1);  
          setCount(prev => prev + 1);  
          // 첫 번째: prev = 현재값 → +1  
          // 두 번째: prev = 방금 업데이트된 값 → 다시 +1  
        }}  
      />  
    )  
  );  
}
```

```
    </View>
  );
}
```

실행해보세요!

빨간 버튼이 진짜 2씩 증가하는 걸 눈으로 확인할 수 있어요!

실습 2: 한 번에 +10 만들기 (더 깔끔한 함수형)

```
jsx

<Button
  title="+10 (한 번에)"
  onPress={() => {
    setCount(prev => prev + 10);    // 한 줄로 끝!
  }}
/>
```

실습 3: 비동기 지연 후에도 정확히 증가 (setTimeout)

```
jsx

<Button
  title="3초 뒤에 +1 (비동기)"
  onPress={() => {
    setTimeout(() => {
      // setCount(count + 1);    // 오래된 count 사용 → 버그!
      setCount(prev => prev + 1);  // 항상 최신값 보장!
    }, 3000);
  }}
/>
```

실습 4: 1초마다 자동 증가 (setInterval)

```
jsx

import { useEffect } from 'react';

useEffect(() => {
  const id = setInterval(() => {
    setCount(prev => prev + 1);  // 이걸로 해야 1초마다 진짜 증가!
  }, 1000);
  return () => clearInterval(id);
}, []);
```

실습 5: 객체 상태 안전하게 업데이트

```
jsx
```



```
const [user, setUser] = useState({ name: '철수', age: 20 });
```

```
<Button  
  title="나이 5살 증가"  
  onPress={() => {  
    // 잘못된 예: 기존 객체 사라짐  
    // setUser({ age: user.age + 5 });  
  
    // 올바른 예: 기존 name 유지하면서 age만 변경  
    setUser(prev => ({ ...prev, age: prev.age + 5 }));  
  }}  
>
```

완전한 실습용 전체 코드 (바로 복사해서 써보세요!)

jsx

```
import React, { useState, useEffect } from 'react';  
import { View, Text, Button, Alert } from 'react-native';  
  
export default function App() {  
  const [count, setCount] = useState(0);  
  const [user, setUser] = useState({ name: '김철수', age: 25 });  
  
  useEffect(() => {  
    const timer = setInterval(() => {  
      setCount(c => c + 1);  
    }, 1000);  
    return () => clearInterval(timer);  
  }, []);  
  
  return (  
    <View style={{ flex: 1, paddingTop: 100, padding: 20 }}>  
      <Text style={{ fontSize: 60, textAlign: 'center' }}>{count}</Text>  
      <Text style={{ fontSize: 24, marginVertical: 20 }}>  
        {user.name} ({user.age}살)  
      </Text>  
  
      <Button title="잘못된 +2 (실제 +1)" onPress={() => {  
        setCount(count + 1);  
        setCount(count + 1);  
      }} />  
  
      <Button title="올바른 +2 (함수형!)" color="#d00" onPress={() => {  
        setCount(c => c + 1);  
        setCount(c => c + 1);  
      }} />  
  
      <Button title="+10" onPress={() => setCount(c => c + 10)} />  
  
      <Button title="3초 뒤 +1" onPress={() => {
```

```

        setTimeout(() => setCount(c => c + 1), 3000);
    }} />

    <Button title="나이 +5" onPress={() => {
        setUser(u => ({ ...u, age: u.age + 5 }));
    }} />

    <Text style={{ marginTop: 30, color: 'gray' }}>
        위에 숫자가 1초마다 자동 증가 중! (함수형 업데이트 덕분)
    </Text>
</View>
);
}

```

이제 직접 실행해보고 버튼 눌러보면

“아하! 이게 함수형 업데이트의 힘인가!” 하고 무릎을 치실 거예요

궁금한 거 더 있으면 바로 말해주세요!

이제 당신은 함수형 업데이트 마스터예요

배치 업데이트 상세 설명

완전 정복해드릴게요!

“배치 업데이트 (Batched Updates)”는 리액트 상태 업데이트 버그의 90% 원인이자, 함수형 업데이트가 왜 필요한지 정확히 이해하게 해주는 핵심 개념입니다.

배치 업데이트란? (한 문장으로)

리액트는 성능을 위해 한 이벤트 핸들러 안에서 일어나는 여러 `setState()`를 모아두었다가, 이벤트가 완전히 끝난 후에 딱 한 번만 리렌더링한다.

이게 왜 문제냐? → “모아두었다가 한 번에 처리”할 때 모든 `setState`는 같은 스냅샷(그 순간의 상태 값)을 기준으로 계산된다!

시각적으로 이해하기 (카운터 예제)

```

jsx

const [count, setCount] = useState(0);

const handleClick = () => {
  console.log('클릭 시작, 현재 count:', count); // 예: 10

  setCount(count + 1); // "10 + 1 → 11로 바꿔줘" 예약
  setCount(count + 1); // "10 + 1 → 11로 바꿔줘" 또 예약 (같은 count 변수!)
  setCount(count + 1); // 또 "10 + 1 → 11"

  console.log('핸들러 끝, 아직 리렌더링 안 됨. count:', count); // 여전히 10!
};

```

```
return <Button title="증가" onPress={handleClick} />;
```

리액트가 실제로 하는 일:

단계	실제 일어나는 일	count 값
1	클릭 발생 → handleClick 실행	10
2	3개의 setCount 예약 (모두 count=10 기준)	아직 10
3	handleClick 함수 완전히 종료	아직 10
4	리액트가 배치 처리 시작 → 가장 마지막 setCount만 적용	11
5	딱 한 번 리렌더링	11

결과: +3을 기대했지만 +1만 됨!

배치 업데이트가 일어나는 정확한 범위

상황	배치 업데이트 적용?	설명
React 이벤트 핸들러 (onClick, onPress 등)	적용됨	가장 중요!
setTimeout / setInterval 안	적용 안 됨	이미 비동기라서
Promise.then() 안	적용 안 됨	비동기
useEffect 안	적용 안 됨	다음 렌더링 이후라서
native 이벤트 (addEventListener)	적용 안 됨	React가 관여 안 함

함수형 업데이트가 배치 업데이트를 뚫는 이유

jsx

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

리액트 내부에서 이렇게 처리됩니다:

JavaScript

```
// 1번째 setCount → 큐에 함수 저장  
queue: [ (prev=10) => 11 ]  
  
// 2번째 setCount → 큐에 또 함수 추가  
queue: [ (prev=10)=>11, (prev=??)=>?? ]
```

```
// 3번째 setCount
queue: [ (10)=>11, (?)=>?, (?)=>? ]
```

```
// 배치 처리 시작! → 큐를 순서대로 실행
step1: prev=10 → 반환 11
step2: prev=11 → 반환 12
step3: prev=12 → 반환 13
```

최종 결과: 13 (정확히 +3!)

즐거운 체인 반응이 일어나는 거예요!

실시간 테스트 코드 (당신이 직접 확인 가능)

```
jsx

import { useState } from 'react';
import { Button, Text, View } from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);

  const badWay = () => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    console.log('badWay 끝, count는?', count); // 여전히 예전 값!
  };

  const goodWay = () => {
    setCount(c => c + 1);
    setCount(c => c + 1);
    setCount(c => c + 1);
    console.log('goodWay 끝, count는?', count); // 여전히 예전 값! (정상)
  };

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text style={{ fontSize: 80 }}>{count}</Text>

      <Button title="나쁜 방법 (+3 하려다 +1)" onPress={badWay} />
      <Button title="좋은 방법 (정확히 +3)" color="red" onPress={goodWay} />

      <Button
        title="비동기라서 배치 안 됨 (정확히 +3)"
        onPress={() => setTimeout(() => {
          setCount(count + 1);
          setCount(count + 1);
          setCount(count + 1);
        }, 100)}
      />
    </View>
  );
}
```

```
    </View>
  );
}
```

눈으로 직접 확인해보세요!

- 첫 번째 버튼 → +1만 됨
- 두 번째 버튼 → +3 정확히 됨
- 세 번째 버튼 → setTimeout 안이라 배치 안 되므로 +3 됨

최종 정리표

상황	배치 업데이트 됨?	count + 1 여러 번 써도 됨?	안전한 방법
onPress 안	됨	안 됨 (+1만 됨)	함수형 업데이트 필수
setTimeout 안	안 됨	됨	둘 다 가능
useEffect 안	안 됨	됨	둘 다 가능
Promise 안에	안 됨	됨	둘 다 가능

이제 배치 업데이트 완전히 정복하셨습니다!

앞으로 “왜 +2 했는데 +1만 되지?” 할 일 절대 없어요

더 깊이 들어가고 싶으면 Concurrent Mode에서의 배치 변화도 알려드릴게요!

↳ Concurrent Mode 배치 변화 설명

↳ useReducer 상태 관리

↳ 설명 더 간결하게